

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA

**ARTHUR ERPEN
CAETANO PERUZZO
GABRIEL HERRERA**

Relatório sobre a Etapa 3 de Sistemas Operacionais II

Florianópolis
2026

Resumo

Este relatório descreve a implementação da Etapa 3 do projeto de Sistemas Operacionais II: a adição de *timestamps* nas mensagens e a sincronização dos relógios entre sistemas autônomos distintos via uma implementação simplificada do Precision Time Protocol (IEEE 1588), chamada **SPTP**. Cada **Message** agora carrega um campo **timestamp** gerado por `Clock::monotonic_stamp` no instante da transmissão; um mecanismo atômico garante que o par (*endereço de origem, timestamp*) seja único em toda a rede. A sincronização inter-VM é realizada exclusivamente no processo *gateway*, que troca quatro timestamps com o nó mestre (**RSU dedicada**), estima o *offset* e aplica a correção via `clock_settime(CLOCK_REALTIME)`. O texto apresenta os requisitos, a arquitetura, os detalhes de implementação e a validação experimental pelos testes `sptp_simple` (6 VMs: RSU + 2 senders + 3 receivers) e os benchmarks de latência inter-VM e intra-VM, executados em QEMU com kernel PREEMPT_RT 6.15.5.

Sumário

1	Introdução	2
2	Formato das mensagens	2
2.1	Timestamp nas mensagens de dados	2
2.2	Unicidade: <code>Clock::monotonic_stamp</code>	3
2.3	Uso de <code>CLOCK_REALTIME</code>	4
2.4	Formato PTP	4
3	Protocolo SPTP	4
3.1	Visão geral	4
3.2	Troca de mensagens e os 4 timestamps	4
3.3	Estimação de offset e delay	5
3.4	Filtro de outliers	5
3.5	Ajuste do relógio (<code>clock_settime</code>)	5
3.6	Recovery rápido pré-sincronização	6
4	Integração no Protocol	6
4.1	Instanciação e parâmetro de compilação	6
4.2	Fluxo de recepção: <code>Protocol::update()</code>	7
4.3	Fluxo de envio: <code>Protocol::send_via_nic()</code>	8
5	Validação experimental	8
5.1	Cenário <code>sptp_simple</code> (6 VMs)	9
5.2	Cenários de latência: <code>measure-rtt</code> e <code>measure-rtt-intra</code>	9
6	Resultados	9
7	Conclusão	11

1 Introdução

Nas etapas anteriores foi construída uma API de comunicação (`Communicator / Protocol / NIC / Engine`) capaz de transportar mensagens dentro de uma VM (via memória compartilhada System V) e entre VMs distintas (via `AF_PACKET`). A `Message` já carregava um campo `Origin = {MAC, porta lógica}`, suficiente para identificar o remetente.

A Etapa 3 acrescenta a dimensão temporal: cada mensagem passa a ser *etiquetada* com o instante em que foi enviada, e os relógios das diferentes VMs devem ser sincronizados de modo que o par (`address, timestamp`) identifique unicamente cada mensagem em toda a rede. Para tanto, implementamos um protocolo simplificado baseado no PTP (IEEE 1588), denominado **SPTP** (*Simplified PTP*).

Requisitos da etapa:

- Mensagens de dados seguem o formato $M = \{address, timestamp, payload\}$.
- Mensagens PTP seguem o formato $PTP = \{address, timestamp, ptp_frame\}$.
- O par (`address, timestamp`) deve ser único na percepção de toda a rede.
- Processos da mesma VM compartilham a percepção de tempo (acesso comum ao relógio do etiquetador).
- Sincronização inter-VM via implementação simplificada do PTP; a escolha de qual agente é o mestre pode ser arbitrária, podendo inclusive ser uma RSU dedicada.

2 Formato das mensagens

2.1 Timestamp nas mensagens de dados

A classe `Message` foi estendida com um campo `int64_t _timestamp`, acessível publicamente apenas por leitura via `timestamp()`. Para escrever o campo sem expor setters a código de aplicação, `Message` declara `Communicator` como `friend`:

```
class Message {
public:
    template <typename Channel>
    friend class Communicator;

    int64_t timestamp() const { return _timestamp; }
    const Vehicle_Protocol::Address & origin() const { return _origin; }
    // ... construtores e data()/size() ...
private:
    Vehicle_Protocol::Address _origin;
    int64_t _timestamp;
    // ...
};
```

O `Communicator::send` escreve diretamente no campo privado antes de delegar para o canal:

```
bool send(Message * message) {
    message->_timestamp = Clock::monotonic_stamp();
    return (_channel->send(_address, Address::logical_broadcast(),
                           message->data(), message->size(),
                           message->_timestamp) > 0);
}
```

Do lado da recepção, `Protocol::receive` extrai o timestamp do cabeçalho do pacote e o deposita em `Message::_timestamp`, de modo que a aplicação o obtenha via `m.timestamp()`.

O cabeçalho do pacote (`Protocol::Header`) carrega o timestamp junto com as portas lógicas e um campo `PacketKind`, que distingue mensagens de dados de mensagens internas do protocolo PTP:

```
class Header {
    int64_t    _timestamp;
    Port       _src_port;
    Port       _dst_port;
    PacketKind _kind;      // DATA, SPTP_SYNC, SPTP_REQUEST_SYNC
};
```

2.2 Unicidade: `Clock::monotonic_stamp`

Para que o par (`address`, `timestamp`) seja único mesmo após um ajuste de relógio pelo SPTP ou no caso improvável de duas mensagens saírem no mesmo nanosegundo, implementamos `Clock::monotonic_stamp`:

```
namespace Clock {
    inline int64_t now_ns() {
        struct timespec ts;
        clock_gettime(CLOCK_REALTIME, &ts);
        return (int64_t)ts.tv_sec * 1'000'000'000LL + ts.tv_nsec;
    }

    inline int64_t monotonic_stamp() {
        static std::atomic<int64_t> last{0};
        int64_t now = now_ns(), prev = last.load(std::memory_order_relaxed), ts;
        do {
            ts = (now > prev) ? now : prev + 1;
        } while (!last.compare_exchange_weak(prev, ts,
                                              std::memory_order_relaxed));

        return ts;
    }
}
```

O `compare_exchange_weak` garante exclusão mútua sem *mutex*: se duas threads chamarem `monotonic_stamp` simultaneamente, apenas uma atualiza `last` por iteração do *loop*; a outra repete a tentativa com o valor atualizado. Esse mecanismo garante que dois timestamps emitidos pelo mesmo agente nunca colidam, mesmo se o relógio for ajustado para trás pelo SPTP.

2.3 Uso de CLOCK_REALTIME

Utilizamos `CLOCK_REALTIME` — e não `CLOCK_MONOTONIC` — por duas razões complementares:

1. `CLOCK_REALTIME` é compartilhado por todos os processos do mesmo sistema operacional, satisfazendo o requisito de “*todos os processos da mesma VM têm acesso ao tempo do etiquetador*”.
2. O SPTP precisa poder corrigir o relógio do *slave* via `clock_settime`; essa chamada de sistema ajusta `CLOCK_REALTIME` mas não afeta `CLOCK_MONOTONIC`.

2.4 Formato PTP

O tráfego PTP não é encapsulado em `Message`; em vez disso, utiliza o campo `PacketKind` do cabeçalho `Protocol::Header` para sinalizar ao `Protocol::update()` que o pacote deve ser consumido internamente e jamais entregue à camada de aplicação. Os *payloads* específicos do PTP são definidos como estruturas internas de `SPTP_Protocol`:

```
struct Request_Payload { uint32_t seq_id; } __attribute__((packed));
struct Sync_Payload    { uint32_t seq_id; int64_t t2_prime; }
                      __attribute__((packed));
```

3 Protocolo SPTP

3.1 Visão geral

O SPTP (*Simplified PTP*) é uma simplificação do IEEE 1588 que opera exclusivamente no processo *gateway*, aproveitando a `RawSocketNIC` já existente. O mestre é fornecido por uma **RSU dedicada** (`src/application/rsu.cpp`): uma VM cujo gateway sobe com `set_master(true)` e não executa nenhum componente de aplicação — apenas responde a requisições de sincronização dos demais. As demais VMs são escravas: seu gateway sobe com `Vehicle(false)` e elas hospedam os componentes do experimento.

O protocolo usa quatro timestamps para estimar, de forma simétrica, o *offset* entre os relógios e o *delay* de propagação unidirecional.

3.2 Troca de mensagens e os 4 timestamps

A Figura 1 ilustra a troca de mensagens.

1. O *slave* envia `REQUEST_SYNC` e captura t_2 no instante mais próximo possível do envio físico.
2. O *master* (RSU) registra t'_2 assim que o pacote chega e responde com `SYNC`, colocando t'_2 no *payload* e carimbando t_1 no campo `timestamp` do cabeçalho (gravado dentro de `send_via_nic`, próximo da NIC).
3. O *slave* registra $t'_1 = \text{Clock::now_ns}()$ ao receber `SYNC` e calcula:

$$\text{offset} = \frac{(t'_1 - t_1) - (t'_2 - t_2)}{2} \quad \text{delay} = \frac{(t'_1 - t_1) + (t'_2 - t_2)}{2}$$

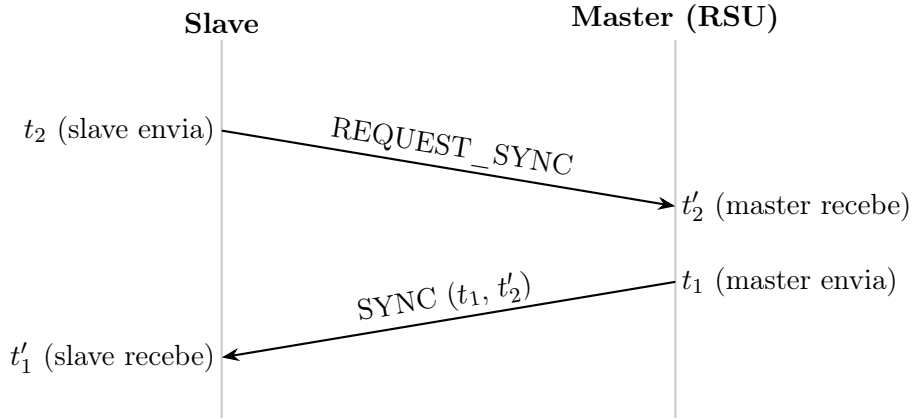


Figura 1: Troca de mensagens SPTP com 4 timestamps.

3.3 Estimação de offset e delay

O campo `seq_id` em ambos os payloads garante que o *slave* descarte respostas tardias de rodadas anteriores. O campo `_pending_t2_ns` é zerado após cada rodada bem-sucedida, impedindo que uma segunda resposta espúria reutilize um t_2 antigo.

O delay é suavizado por uma EWMA com fator $\alpha = 0,125$:

```

if (raw_delay > 0)
    _current_delay_ns = (1.0 - Cfg::ALPHA)*_current_delay_ns
                      + Cfg::ALPHA * raw_delay;

```

Apenas ajustes dentro da faixa $[100\mu s, 1s]$ são aplicados, evitando tanto correções ruidosas (offset minúsculo) quanto saltos absurdos (offset maior que o esperado, indício de pacote corrompido ou fora de ordem).

3.4 Filtro de outliers

Medições com $|\text{offset}| \geq 1s$ são tratadas como outlier — tipicamente ruído de boot caótico ou caminho congestionado. Nesse caso o slave **não** marca progresso (`_last_sync_steady_ns` e `_first_sync_done` permanecem inalterados), e o *watchdog* continua tentando até uma medição plausível:

```

int64_t abs_offset = std::llabs(raw_offset);
if (abs_offset >= Cfg::MAX_OFFSET_NS) {
    _pending_t2_ns.store(0, std::memory_order_release);
    return; // outlier: nao progride o watchdog
}
_last_sync_steady_ns.store(steady_now_ns(), std::memory_order_release);
_first_sync_done.store(true, std::memory_order_release);
if (abs_offset > Cfg::MIN_OFFSET_NS) adjust_clock_jump(-raw_offset);

```

3.5 Ajuste do relógio (clock_settime)

A correção é aplicada como *salto* direto sobre `CLOCK_REALTIME`:

```

void adjust_clock_jump(int64_t delta_ns) {
    struct timespec ts;

```

```

clock_gettime(CLOCK_REALTIME, &ts);
int64_t total = (int64_t)ts.tv_sec * 1'000'000'000LL
                + ts.tv_nsec + delta_ns;
ts.tv_sec = total / 1'000'000'000LL;
ts.tv_nsec = total % 1'000'000'000LL;
if (clock_settime(CLOCK_REALTIME, &ts) < 0)
    perror("[SPTP] clock_settime");
}

```

Como `CLOCK_REALTIME` é compartilhado por todos os processos da VM, o ajuste feito pelo gateway propaga-se instantaneamente para todos os componentes — incluindo `Clock::now_ns()` chamado no momento em que um componente etiqueta uma mensagem.

3.6 Recovery rápido pré-sincronização

O slave inicia uma sincronização imediatamente ao ligar (`start()` chama `send_sync_request()` antes de criar a thread). Se o `REQUEST_SYNC` ou o `SYNC` se perderem (cenário plausível quando várias VMs bootam simultaneamente e disparam broadcasts no mesmo instante), esperar o intervalo nominal de `MAX_SILENCE_S` (15s) antes de retentar deixaria o slave sem sincronização por uma fração significativa do teste. Para cobrir esse caso, o *watchdog* faz *retry rápido* a cada `INITIAL_RETRY_MS` (250ms) enquanto a flag atômica `_first_sync_done` permanecer `false`; somente após a primeira medição aceita ele entra no modo de polling longo:

```

_silence_worker = std::thread([this]() {
    while (_running.load(std::memory_order_acquire)) {
        if (!_first_sync_done.load(std::memory_order_acquire)) {
            std::this_thread::sleep_for(milliseconds(Cfg::INITIAL_RETRY_MS));
            if (_running.load(std::memory_order_acquire)
                && !_first_sync_done.load(std::memory_order_acquire)) {
                send_sync_request();
            }
            continue;
        }
        std::this_thread::sleep_for(duration<double>(_max_silence_s));
        // ... checagem normal de inatividade ...
    }
});

```

O `steady_clock` (imune a `clock_settime`) é utilizado exclusivamente para medir o intervalo de silêncio, evitando que um ajuste do SPTP dispare uma ressincronização espúria.

4 Integração no Protocol

4.1 Instanciação e parâmetro de compilação

O `Protocol` é um template parametrizado por dois tipos de NIC: `Protocol<ShmNIC, RawSocketNIC>`. No gateway, ambos são concretos; nos componentes de aplicação (Sender, Receiver etc.), `RawSocketNIC` é `void`. O `SPTP_Protocol` só existe quando `RawSocketNIC` não é `void`, garantido pelo bloco `if constexpr`:

```

void enable_sync(bool is_master) {
    if constexpr (!std::is_void_v<RawSocketNIC>) {
        if (!_socket_nic || !_sptp) return;
        _is_master = is_master;
        Address own_addr(_socket_nic->address(), 0);
        _sptp = new SPTP_Protocol<Address>(own_addr, _is_master,
                                           &Protocol::send_via_nic);

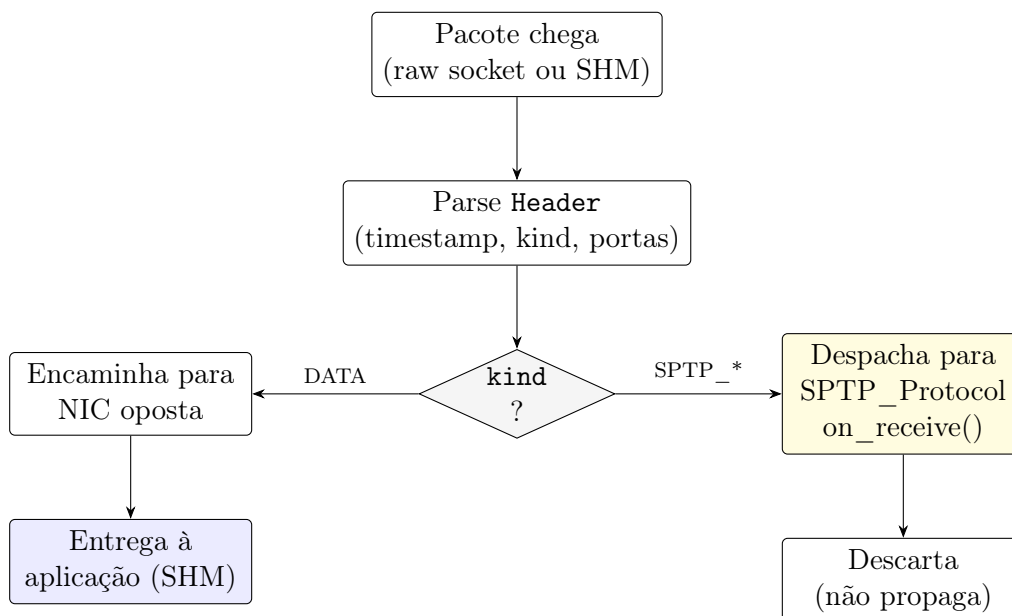
        _sptp->start();
    }
}

```

Em tempo de compilação, o bloco é completamente eliminado para os componentes comuns — nenhum código SPTP existe nesses processos. O gateway chama `enable_sync(true)` na RSU e `enable_sync(false)` nos demais veículos.

4.2 Fluxo de recepção: `Protocol::update()`

`Protocol::update()` é invocado ciclicamente pela thread principal do gateway para drenar ambas as NICs. O diagrama a seguir resume os caminhos possíveis para um pacote que chega pela raw socket (rede externa) ou pela SHM (componentes locais):



O campo `PacketKind` no cabeçalho é o ponto de bifurcação central:

- **DATA:** o pacote é encaminhado pela NIC oposta (raw→SHM ou SHM→raw) e, se o destino for local, entregue à aplicação com o timestamp preservado do cabeçalho.
- **SPTP_REQUEST_SYNC / SPTP_SYNC:** o pacote é passado a `_sptp->on_receive()` e descartado em seguida, sem jamais entrar na fila da SHM. A aplicação nunca vê tráfego SPTP.

Em código:

```
// Protocol::update() -- trecho de despacho
if (kind == PacketKind::SPTP_REQUEST_SYNC
    || kind == PacketKind::SPTP_SYNC) {
    if (_sntp) _sntp->on_receive(sender_addr, kind,
                                payload, payload_size);

    return; // nao propaga
}
// caminho DATA: encaminha / entrega
forward_or_deliver(packet);
```

4.3 Fluxo de envio: Protocol::send_via_nic()

Quando a aplicação chama Communicator::send, o caminho é:

Communicator::send → Protocol::send (SHM) → Protocol::send_via_nic (raw)

Em send_via_nic, o timestamp é gravado no cabeçalho imediatamente antes de entregar o buffer para a NIC:

```
// Protocol::send_via_nic (trecho)
int64_t now = Clock::now_ns();
packet->timestamp(ts != 0 ? ts : now);
return nic->send(buf);
```

O SPTP_Protocol envia seus próprios pacotes de controle chamando diretamente send_via_nic (passado como callback na construção), reutilizando a mesma raw socket sem nenhuma camada intermediária. O PacketKind diferente garante que esses pacotes sejam descartados pelo update() de qualquer gateway que os receba, inclusive o próprio master.

A Figura 2 resume a posição estrutural do SPTP na arquitetura.

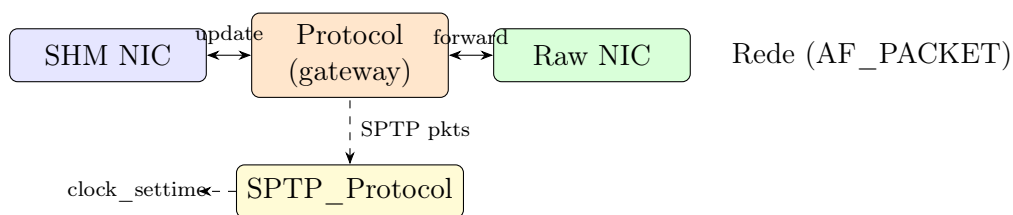


Figura 2: Posição do SPTP_Protocol na arquitetura do gateway.

5 Validação experimental

A Etapa 3 é validada por três cenários complementares:

- **sntp_simple**: 6 VMs, mede o offset residual entre o relógio dos slaves e o timestamp do transmissor para cada pacote recebido. Executado pelo alvo **test** do **make default**.

- **measure-rtt**: 5 VMs, mede a latência de ida inter-VM via raw socket (RTT/2).
- **measure-rtt-intra**: 1 VM, mede a latência de ida intra-VM via SHM (RTT/2).

O cenário **sptp_simple** usa uma RSU dedicada como master SPTP. Os componentes de aplicação rodam como processos filhos do gateway (via **fork**), sem política de escalonamento especial — o gateway em si e suas threads de recepção (raw socket e SHM) são elevados a **SCHED_FIFO** para garantir timestamps precisos.

5.1 Cenário sptp_simple (6 VMs)

Topologia:

- VM1: RSU (master SPTP, sem componentes de aplicação).
- VM2, VM3: **Vehicle** slave + **Sender** (dois transmissores defasados em 250 ms dentro do ciclo de 500 ms).
- VM4–VM6: **Vehicle** slave + **Receiver** (medem o offset por pacote recebido, separando amostras por sender de origem).

A RSU sobe primeiro e responde a **REQUEST_SYNC** dos slaves. Cada **Sender** aguarda 10 s (**STARTUP_DELAY_S**) para garantir que todas as VMs já sincronizaram; o **Sender B** aguarda 250 ms adicionais para defasar os dois fluxos dentro do mesmo ciclo. Em seguida, cada sender transmite mensagens em broadcast com intervalo de 500 ms. Cada **Receiver** acumula amostras por sender, descarta as 3 primeiras (transitório de sincronização) e calcula sobre as restantes:

$$\delta_{us} = \frac{\text{Clock::now_ns()}_{\text{slave}} - \text{sender_msg_ts}}{1000}$$

que é a soma de três componentes: **delay_propagacao** + **offset_residual_pos_sync** + **drift_acumulado**.

A defasagem entre os dois senders demonstra a *identificação inequívoca* exigida pela etapa: receivers distinguem as mensagens pelo par (**origin**, **timestamp**), atribuindo cada amostra ao sender correto mesmo quando dois pacotes chegam com poucos milissegundos de diferença.

5.2 Cenários de latência: measure-rtt e measure-rtt-intra

O benchmark **measure-rtt** (5 VMs, raw socket) e o **measure-rtt-intra** (1 VM, SHM) medem a latência de ida estimada como RTT/2, fornecendo a referência quantitativa do **delay_propagacao** que compõe o residual medido pelo **sptp_simple**.

6 Resultados

As tabelas a seguir apresentam os resultados obtidos no run de 10/05/2026, em QEMU sobre kernel próprio PREEMPT_RT 6.15.5.

Análise dos resultados:

Tabela 1: Cenário `sptp_simple`: residual $\delta = \text{slave_now} - \text{sender_ts}$ (em μs , 22 amostras estáveis por linha após descarte do transitório).

Receiver	Sender	Amostras	Min (μs)	Max (μs)	$ \bar{\delta} $ (μs)
vm4	vm2	22	1 670	4 633	2 947
vm4	vm3	22	1 519	6 644	3 324
vm5	vm2	22	-1 174	2 800	1 776
vm5	vm3	22	-1 679	2 333	1 763
vm6	vm2	22	-614	2 935	1 530
vm6	vm3	22	-1 154	2 247	1 837

Tabela 2: Latência de ida estimada (RTT/2) inter-VM (`measure-rtt`, 5 VMs, raw socket) e intra-VM (`measure-rtt-intra`, 1 VM, SHM).

Cenário	Amostras	Média (μs)	Mín (μs)	Máx (μs)
Inter-VM (raw socket)	20	4 123	1 396	43 626
Intra-VM (SHM)	5 000	461	410	4 928

- **sptp_simple**: o residual $|\bar{\delta}|$ ficou entre 1.5 ms e 3.3 ms. Esse valor é a soma de três componentes: delay de propagação inter-VM, offset residual pós-sincronização e drift acumulado durante a janela de envio. Valores negativos de mínimo (vm5 e vm6) indicam que o relógio do receiver ficou momentaneamente *à frente* do sender após a convergência do SPTP — comportamento esperado dada a simetria da fórmula de offset e a variabilidade de escalonamento do QEMU. A Tabela 2 fornece a referência para isolar a componente de delay: a latência média inter-VM (raw socket) foi 4 123 μs , consistente com o piso positivo observado em vm4 (mín $\approx 1.5\text{--}1.7\text{ ms} \approx$ latência mínima de 1 396 μs + offset residual). Nos pares vm5 e vm6 o offset residual é menor que o delay, fazendo o δ instantâneo oscilar em torno de zero.
- **Identificação inequívoca**: com dois senders transmitindo mensagens defasadas em 250 ms dentro do mesmo ciclo de 500 ms, os receivers distinguiram corretamente cada origem pelo par (`address`, `timestamp`), sem confundir amostras de senders distintos — demonstrando o requisito central da etapa.
- **Latência de comunicação**: o benchmark de RTT (Tabela 2) contextualiza as medições SPTP. A latência de ida intra-VM via SHM é de 461 μs em média (mín 410 μs), cerca de 9 $\times$ menor que a latência inter-VM via raw socket (4 123 μs em média). A dispersão do RTT inter-VM (máx 43 ms) reflete o jitter de escalonamento do QEMU, que também é a principal fonte de variância no offset medido pelo `sptp_simple`.
- **Compartilhamento intra-VM**: como `CLOCK_REALTIME` é por-kernel e cada VM tem o seu, qualquer processo dentro da mesma VM observa o mesmo relógio. Quando o gateway aplica `clock_settime`, a correção se propaga instantaneamente para todos os processos componentes daquela VM — requisito da etapa atendido por construção.

7 Conclusão

A Etapa 3 acrescentou a dimensão temporal ao sistema, cumprindo os dois requisitos centrais: etiquetagem de mensagens com timestamp único e sincronização dos relógios entre VMs. A solução reutiliza integralmente a infraestrutura de comunicação das etapas anteriores — o SPTP opera no processo gateway, aproveitando a `RawSocketNIC` existente, e os pacotes de controle são filtrados em `Protocol::update()` antes de chegar à aplicação. A introdução da `RSU` como master dedicado formaliza o papel do agente sincronizador sugerido pelo enunciado.

Duas melhorias de corretude foram implementadas além da especificação básica do PTP de 4 timestamps: (i) o *retry rápido* pré-sincronização (250 ms enquanto `_first_sync_done == false`) cobre perdas do `REQUEST_SYNC` ou `SYNC` inicial sem fazer o slave esperar 15 s; e (ii) o filtro de outliers (`abs_offset >= MAX_OFFSET_NS = 1 s`) impede que medições espúrias — típicas durante o boot caótico de múltiplas VMs simultâneas — travem o relógio em valor errado.

Os testes automatizados em QEMU confirmaram que o `CLOCK_REALTIME` é efetivamente compartilhado intra-VM (por construção via kernel) e que o SPTP converge para offsets inter-VM da ordem de 1.5 a 3.3 ms em média. Os benchmarks de RTT (Tabela 2) mostram que a maior parte desse residual é explicada pela própria latência de propagação da raw socket (média 4.1 ms, mínimo 1.4 ms), confirmando a corretude da sincronização.